



SMARTSTOCK • DOCUMENTAÇÃO TÉCNICA

SmartStock Entregas

Especificação funcional, arquitetura Flutter, persistência local e plano de integração com Spring Boot.

Versão: 1.0.0

Plataforma: Flutter / Android e iOS

Modo inicial: local-first com SQLite

Data: agosto de 2026

1. Visão geral e objetivo

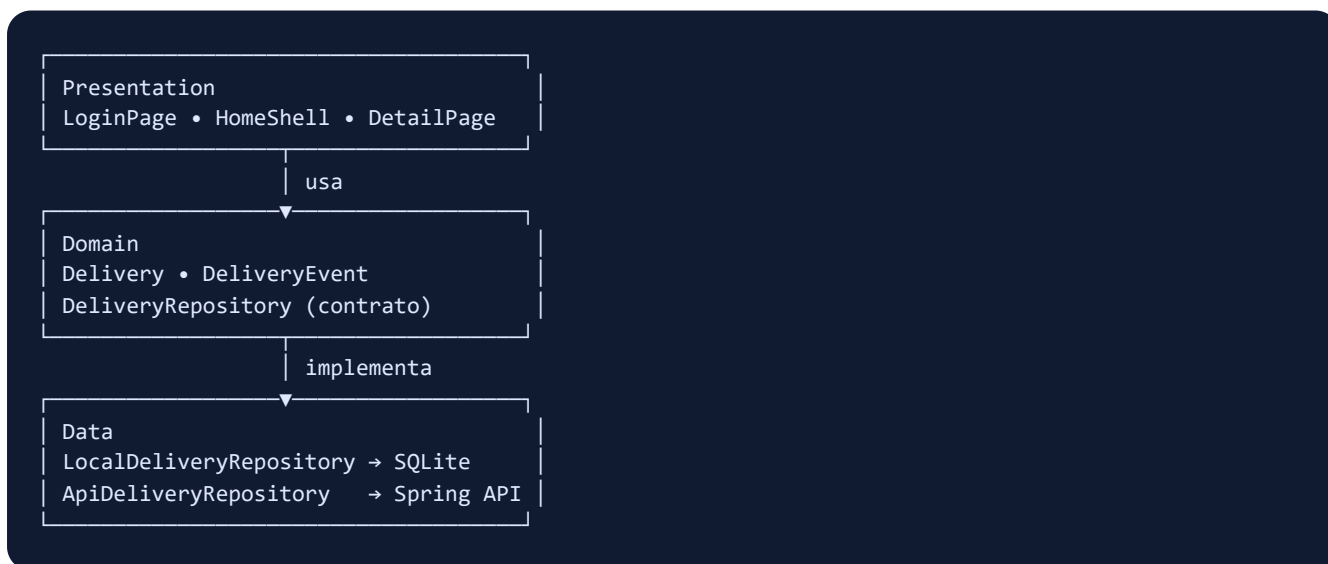
O SmartStock Entregas é um aplicativo móvel para a última etapa do processo comercial: a entrega. Ele foi feito como produto independente do ERP e do backend atual. Na primeira versão roda com dados locais persistentes; na próxima etapa pode integrar qualquer backend Java/Spring Boot por API REST.

Princípio principal: as telas não conhecem SQLite ou HTTP. Elas dependem do contrato `DeliveryRepository`, permitindo trocar a fonte de dados sem reescrever a experiência do entregador.

2. Escopo funcional implementado

Área	Implementação
Autenticação	Login local demonstrativo, preparado para substituição por JWT.
Rota	Rota diária, lista ordenada, filtros por status, busca e próxima parada.
Entrega	Detalhe de cliente, endereço, pedido, itens e observações.
Ações externas	Google Maps, discador e WhatsApp por meio do sistema operacional.
Finalização	Iniciar rota, confirmar entrega, nome do recebedor, foto e ocorrência.
Offline	SQLite, eventos locais e contador de pendências de sincronização.

3. Arquitetura de software



Foi utilizada uma organização inspirada em Clean Architecture: modelos e contratos no domínio, implementações de acesso em data e widgets em features.

4. Estrutura do projeto

```

lib/
├─ app/           # MaterialApp, tema e composição
├─ data/
│   └─ local/     # Repositório SQLite e seed demonstrativo
├─ domain/        # Modelos e contrato de repositório
└─ features/
    ├─ auth/      # Login
    ├─ home/      # Início, entregas, rota e perfil
    └─ deliveries/ # Detalhe, comprovante e ocorrência
docs/            # HTML e PDFs de documentação

```

5. Modelo de domínio e regras

Modelo	Responsabilidade
AppUser	Usuário logado: identificador, nome e e-mail.
Delivery	Pedido, cliente, endereço, sequência, status, itens e evidências.
DeliveryItem	Item e quantidade pertencentes à entrega.
DeliveryEvent	Histórico de alteração de status, data e observação.

```

pending → onRoute → delivered
                        ↳ failed

```

Uma entrega finalizada não retorna ao fluxo normal no aplicativo. O servidor futuro deve validar transições e armazenar todo evento para auditoria.

6. Persistência e operação offline

O banco `smartstock_delivery.db` é criado pelo pacote `sqflite` dentro da área privada do aplicativo.

Tabela	Uso
deliveries	Entrega serializada em JSON e data de atualização.
delivery_events	Evento de status com campo <code>synced</code> ; forma a fila de operações.

1. Ao atualizar uma entrega, o app grava o novo estado em `deliveries`.
2. Na mesma transação, cria um evento pendente em `delivery_events`.
3. A interface mostra a quantidade de atualizações aguardando servidor.
4. Na integração futura, um serviço enviará os eventos em ordem e marcará cada item como sincronizado.

Limite atual: a fila é persistida e visível; o envio HTTP automático será ligado quando o backend Spring Boot fornecer os endpoints definidos nesta documentação.

7. Dependências e permissões

Pacote	Uso
sqlite + path	Banco relacional local e caminho seguro do arquivo.
image_picker	Captura da foto do comprovante.
url_launcher	Abertura de Maps, telefone e WhatsApp.

O manifesto Android declara a permissão `android.permission.CAMERA`. A câmera deve ser solicitada somente no momento de anexar um comprovante.

8. Contrato de integração com Spring Boot

Método	Endpoint	Uso
POST	<code>/api/mobile/auth/login</code>	Login; retorna access token e refresh token.
POST	<code>/api/mobile/auth/refresh</code>	Renovação da sessão.
GET	<code>/api/mobile/routes/today</code>	Rota e paradas do entregador autenticado.
GET	<code>/api/mobile/deliveries/{id}</code>	Detalhe atualizado de entrega.
POST	<code>/api/mobile/deliveries/{id}/events</code>	Início, conclusão ou ocorrência.
POST	<code>/api/mobile/deliveries/{id}/proofs</code>	Nome do recebedor e arquivo do comprovante.
POST	<code>/api/mobile/sync</code>	Envio em lote de eventos feitos offline.

```
{
  "eventId": "uuid-gerado-no-celular",
  "status": "DELIVERED",
  "occurredAt": "2026-08-21T14:30:00Z",
  "note": "Recebido por Maria"
}
```

O campo `eventId` deve ser idempotente: se o aparelho reenviar o mesmo evento, o backend não pode duplicar a alteração.

9. Segurança e LGPD para produção

- JWT curto com refresh token e HTTPS obrigatório.
- Senha armazenada com BCrypt ou Argon2 no backend.
- O servidor filtra as entregas pelo entregador autenticado; o Flutter não define a autorização.
- Fotos ficam fora do banco, com acesso protegido.
- Auditar status, usuário, data e origem da alteração.
- Solicitar localização apenas com consentimento e finalidade clara.

10. Execução, testes e próximos passos

```
flutter pub get
flutter analyze
flutter test
flutter run
```

Testes manuais recomendados: login, filtros, início de rota, navegação, ligação, WhatsApp, confirmação com foto, ocorrência, fechamento e reabertura do app para validar a persistência.

Prioridade	Próximo incremento
Alta	Implementar ApiDeliveryRepository, JWT e endpoints do Spring Boot.
Alta	Sincronização automática e resolução de conflitos.
Média	Assinatura digital, galeria de comprovantes e notificações.
Média	GPS consentido, check-in geográfico e rastreamento em rota.
Futura	Mapa interno, ETA e otimização automática da sequência.